

Task 2: Multi-class Classification using a Fully Connected Neural Network

Fashion-MNIST

We use five selected Fashion-MNIST classes and compares three fully connected neural network architectures trained with six optimization methods.

Objective

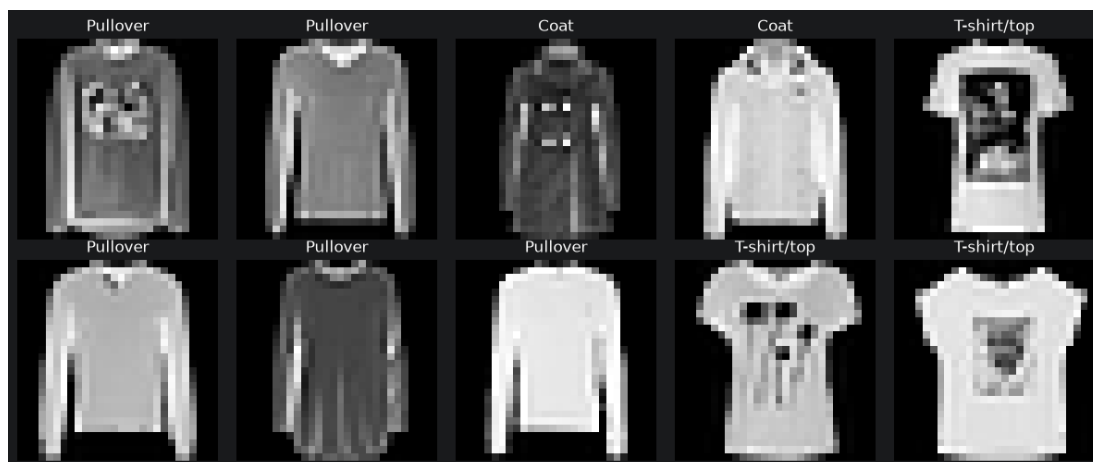
We study how network depth and different backpropagation optimizers affect convergence and classification performance. The comparison was made using the same five classes, the same data split, learning rate, random seed, and initial weights for each architecture.

Dataset and preparation

The experiment used the Fashion-MNIST dataset. The five selected classes were T-shirt/top, Trouser, Pullover, Dress, and Coat. The notebook combined the original training and test portions before selecting these classes, giving 35,000 samples in total, with 7,000 samples in each class.

Class	Samples
T-shirt/top	7,000
Trouser	7,000
Pullover	7,000
Dress	7,000
Coat	7,000
Total	35,000

Pixel values were converted to floating point values and normalized to $[0, 1]$. Each 28×28 grayscale image was flattened into a 784-dimensional input vector. The final stratified split was 22,400 training samples, 5,600 validation samples, and 7,000 test samples, corresponding to 64%, 16%, and 20% of the selected dataset.



Procedure

1. Select the five Fashion-MNIST classes and discard the remaining classes.
2. Normalize the image pixels and flatten each image from 28×28 to 784 features.
3. Create a stratified 64:16:20 training-validation-test split.
4. Construct fully connected networks with ReLU activation in the hidden layers and five output units.
5. Train each architecture using SGD, Batch GD, Momentum, NAG, RMSProp, and Adam.
6. Use cross-entropy loss with a learning rate of 0.001. Momentum and NAG use 0.9; RMSProp uses $\beta=0.99$ and $\epsilon=10^{-8}$; Adam uses $\beta_1=0.9$, $\beta_2=0.999$ and $\epsilon=10^{-8}$.
7. Use the same initial weights for the different optimizers within an architecture.
8. Stop training when the absolute difference between successive average errors falls below 10^{-4} , with a maximum of 100 epochs.
9. Select the best model using validation accuracy and evaluate that model on the held-out test set.

Network architectures

Three architectures were tested. All networks have 784 input features and five output classes; the numbers below refer to the hidden-layer sizes.

Architecture	Hidden layers
Architecture 1	128 – 64 – 32
Architecture 2	256 – 128 – 64 – 32
Architecture 3	256 – 128 – 64 – 32 – 16

Experimental results

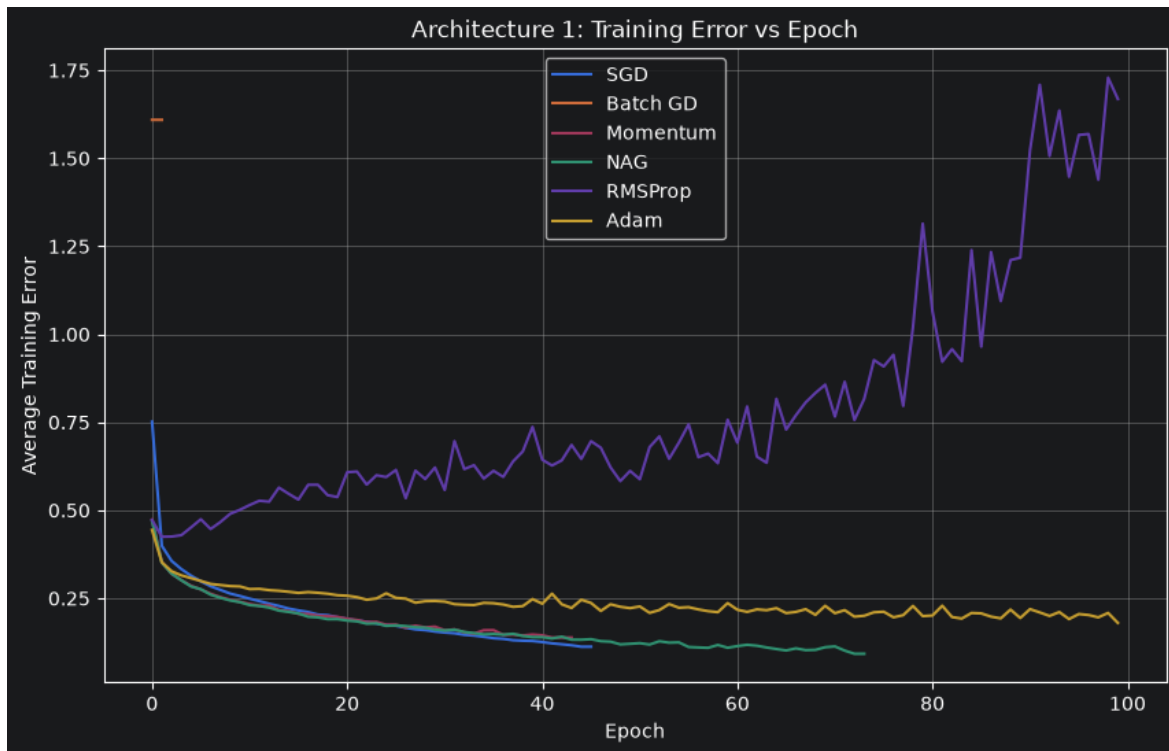
Architecture	Optimizer	Epochs	Train Acc.	Val. Acc.
Architecture 1	SGD	46	96.48%	91.25%
Architecture 1	Batch GD	2	21.09%	20.66%
Architecture 1	Momentum	44	95.24%	91.23%
Architecture 1	NAG	74	95.40%	90.63%
Architecture 1	RMSProp	100	53.24%	52.29%
Architecture 1	Adam	100	95.30%	90.52%
Architecture 2	SGD	57	97.61%	91.41%
Architecture 2	Batch GD	2	19.65%	19.50%
Architecture 2	Momentum	40	93.88%	90.20%
Architecture 2	NAG	96	97.51%	92.11%
Architecture 2	RMSProp	100	40.67%	40.04%
Architecture 2	Adam	100	94.80%	90.54%
Architecture 3	SGD	61	97.49%	91.39%
Architecture 3	Batch GD	2	20.00%	20.00%
Architecture 3	Momentum	27	93.36%	90.89%

Architecture 3	NAG	100	97.19%	91.11%
Architecture 3	RMSProp	53	20.00%	20.00%
Architecture 3	Adam	100	75.47%	74.23%

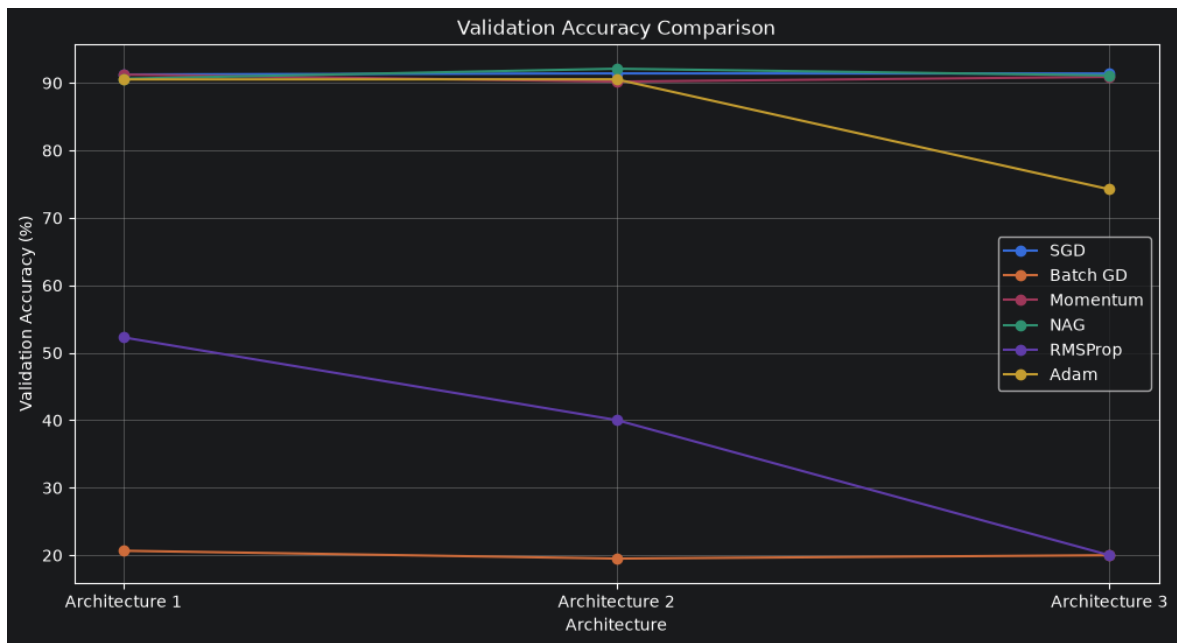
Architecture 2 with NAG gave the highest validation accuracy, 92.11%, and was therefore selected as the best model. Its recorded convergence was at epoch 96.

Training error and validation accuracy plots

The training-error plots show that SGD, Momentum and NAG generally reduce the error as training progresses. In contrast, Batch GD and RMSProp show unstable or ineffective behavior in several runs. Adam also shows a slower and less stable reduction for these experiments.



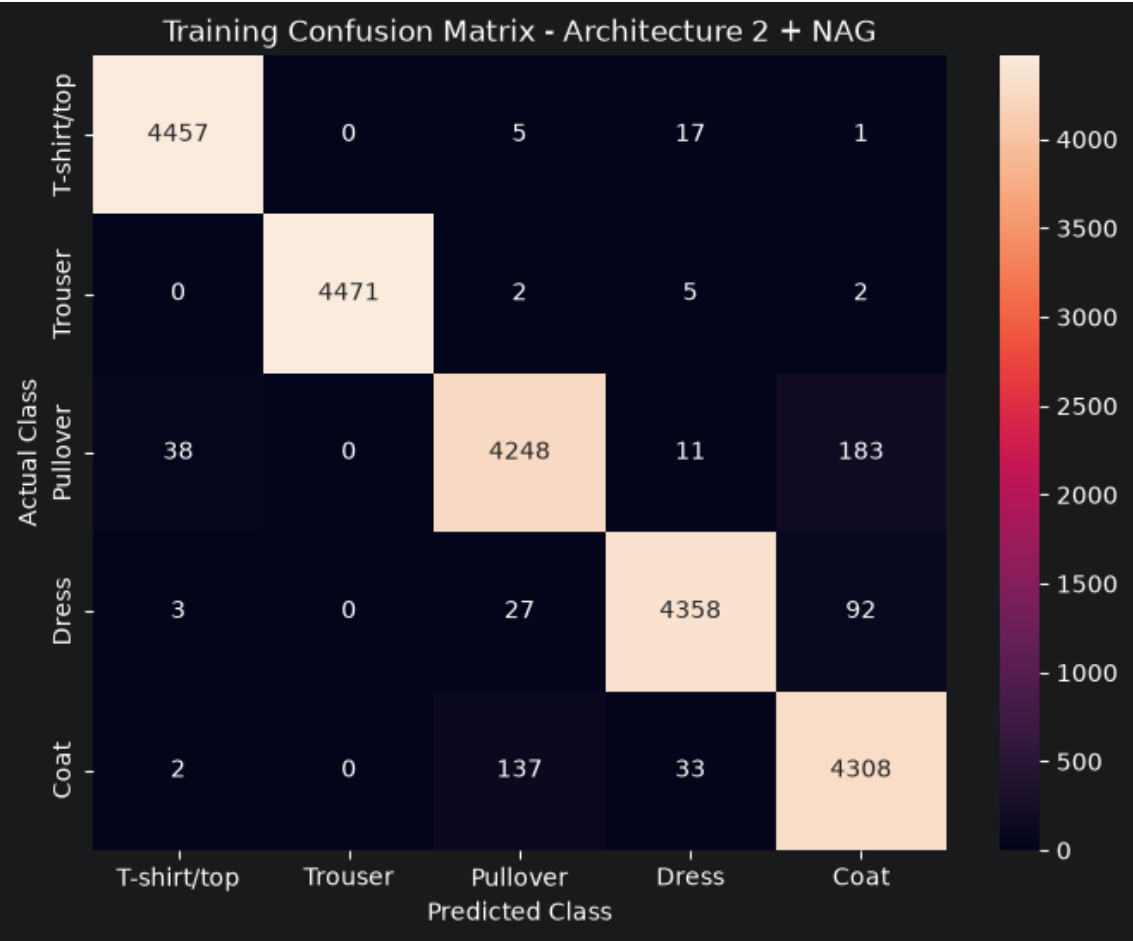


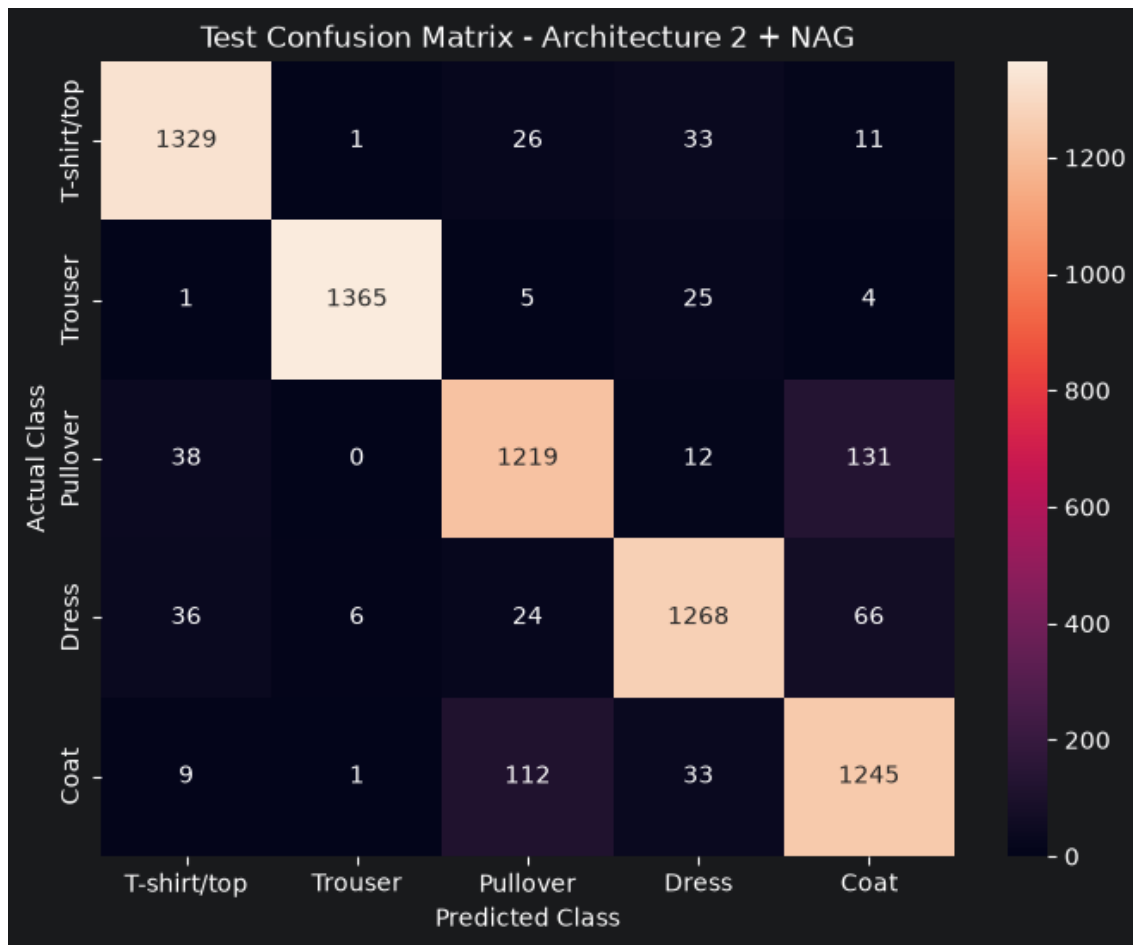


Best model evaluation

The best model was Architecture 2 (256–128–64–32) trained with NAG. Its training accuracy was 97.51%, validation accuracy was 92.11%, and test accuracy was 91.80%. The difference between validation and test accuracy is small, indicating that the model generalized reasonably well to the held-out test set.

Measure	Result
Best architecture	Architecture 2: 256–128–64–32
Optimizer	NAG
Convergence epoch	96
Training accuracy	97.51%
Validation accuracy	92.11%
Test accuracy	91.80%





From the test confusion matrix, the model classified Trouser particularly well. The largest confusion is between Pullover and Coat, which is consistent with the visual similarity of these two clothing categories.

Class	Precision	Recall	F1-score
T-shirt/top	0.94	0.95	0.94
Trouser	0.99	0.97	0.98
Pullover	0.88	0.87	0.88
Dress	0.92	0.91	0.92
Coat	0.85	0.89	0.87

Inference from the results

- The choice of optimizer had a stronger effect on the result than simply increasing the number of hidden layers.
- SGD was consistently strong across the three architectures, reaching validation accuracies around 91.25–91.41%.
- NAG produced the best validation result overall with Architecture 2, reaching 92.11%.
- Batch GD converged in only two epochs in the recorded runs, but the accuracy stayed close to chance level, so early convergence did not mean useful learning.
- RMSProp performed poorly in these particular experiments, especially for Architectures 2 and 3. Its error curves also became unstable.

- Architecture 3 did not improve validation accuracy over Architecture 2. Adding another hidden layer therefore did not provide a benefit for this dataset and these settings.
- The best model has a higher training accuracy than validation accuracy, showing some overfitting, but the test accuracy of 91.80% remains close to the validation accuracy.
- The confusion matrix shows that Trouser is the easiest class for the model, while Coat and Pullover are more difficult and are confused with one another more often.

Conclusion

The experiments show that a fully connected neural network can classify the selected Fashion-MNIST classes with good accuracy. Among the tested combinations, Architecture 2 (256–128–64–32) with NAG gave the best validation accuracy of 92.11% and achieved 91.80% on the test set. The results also show that a deeper network is not automatically better: Architecture 3 did not outperform Architecture 2. For the given implementation and hyperparameters, NAG and SGD were the most effective optimizers, while Batch GD and RMSProp were not effective.